Universidad Tecnológica Nacional — Facultad Regional Mar del Plata
Tecnicatura Universitaria en Programación
Programación IV · Trabajo Práctico Final



Documentación del frontend

Arquitectura, cumplimiento de la consigna y decisiones de diseño

Autor	Eloy Roldán
Proyecto	AlojaME — plataforma de alquileres temporarios
Repositorio	github.com/EloyRolda/tp_gestion_alojamientos
Versión y fecha	1.0 · Noviembre de 2026

Índice

1 Introducción **2**

1.1 Perfiles de usuario 2

2 Cumplimiento de la consigna **3**

3 Arquitectura **4**

3.1 Visión general 4

3.2 Organización del código 4

3.3 Principios de diseño del código 5

4 Autenticación y protección de rutas **6**

5 Decisiones de diseño **7**

5.1 Búsqueda y filtros en el navegador 7

5.2 Calendario de rango para reservar 7

5.3 Fechas siempre en hora local 7

5.4 Una única carta para confirmaciones y avisos 7

5.5 Reglas de cancelación explicadas en pantalla 7

5.6 Pagos simulados para la demostración 7

5.7 Baja de usuarios en cascada 8

5.8 Ficha de usuario para el administrador 8

5.9 Mensajes de error en lenguaje claro 8

5.10 Sin librerías de componentes 8

6 Diseño visual **9**

7 Diseño responsive **10**

8 Limitaciones conocidas **11**

9 Cómo probarlo **12**

1 Introducción

AlojaME es una plataforma web de publicación y reserva de alojamientos temporarios con un modelo de *marketplace*: los anfitriones publican sus propiedades, los clientes las reservan y las pagan a través de la plataforma, y la plataforma retiene el dinero y se lo liquida al anfitrión cuando termina la estadía.

La plataforma no está limitada a una ciudad: cualquier anfitrión puede publicar un alojamiento en cualquier dirección. Los datos de demostración, en cambio, se concentran en Mar del Plata a propósito, para poder mostrar muchos alojamientos en una misma zona y así probar el mapa, los filtros de búsqueda y las estadísticas con un volumen representativo.

Este documento describe el frontend, desarrollado íntegramente en Angular 22: qué se construyó, cómo está organizado y por qué se tomó cada decisión de diseño. Se complementa con el *README* del repositorio (puesta en marcha y despliegue), el *Manual de marca* (identidad visual) y la *Bitácora técnica* (errores encontrados y su solución).

1.1 Perfiles de usuario

Cada perfil accede a una aplicación distinta, con su propio menú y sus propias pantallas.

Perfil	**Funcionalidad disponible**
Visitante (sin cuenta)	Navegar el catálogo: búsqueda con filtros, mapa, detalle del alojamiento, fotos y reseñas.
Cliente	Reservar con un calendario de fechas, pagar con MercadoPago, cambiar fechas, cancelar, dejar reseñas y consultar sus gastos.
Anfitrión	Publicar y editar alojamientos (fotos, servicios y ubicación en el mapa), confirmar y finalizar reservas, cargar su alias de cobro y consultar sus cobros e ingresos.
Administrador	Gestionar usuarios (con una ficha por usuario), los catálogos de tipos y servicios, todas las reservas, los reembolsos manuales, el flujo de dinero y las estadísticas globales.

2 Cumplimiento de la consigna

La siguiente tabla relaciona cada requisito de la consigna de Programación IV con el lugar del proyecto donde se cumple.

Requisito	**Dónde se cumple**
Proyecto íntegramente en Angular 22	Carpeta `frontend/`: Angular 22.2 con componentes *standalone*, sin `NgModules`.
Al menos dos CRUD sobre entidades distintas	Cinco: tipos de alojamiento y servicios (administrador), alojamientos (anfitrión), usuarios (administrador) y reservas (los tres roles).
Login con distintos roles	`AuthService`: el backend emite un JWT y el rol se lee del token. Roles: cliente, anfitrión y administrador.
Guards por rol	`authGuard`, `roleGuard`, `soloVisitanteGuard` e `idNumericoGuard`, aplicados en `app.routes.ts`.
Comunicación HTTP con el backend de Programación III	Todos los servicios de `core/services/` usan `HttpClient` contra la API Spring Boot del proyecto (`GA/`). No se usan json-server ni datos simulados.
Funcionalidad elevada	Pagos con MercadoPago, mapa, calendario de reservas, cancelaciones con reembolso, baja en cascada, ficha de usuario y estadísticas.
Diseño visual destacado	Identidad propia documentada en el *Manual de marca*: paleta, logotipo, mascota Kaza y componentes. Gráficos con Chart.js.
Completamente responsive	Diseño *mobile-first* verificado en 360, 768, 1024 y 1280 píxeles (sección 6).
Documentación y README	Este documento, el *README*, el *Manual de marca*, la *Bitácora técnica* y el *Plan de desarrollo del frontend*.

3 Arquitectura

3.1 Visión general

La aplicación Angular se ejecuta en el navegador y se comunica por HTTPS con el backend Spring Boot desarrollado en Programación III. En el servidor, Nginx entrega los archivos compilados del frontend y reenvía las peticiones `/api` al backend. El backend persiste los datos en MariaDB y se integra con tres servicios externos: MercadoPago (cobros), Cloudinary (imágenes) y Brevo (correo de verificación).

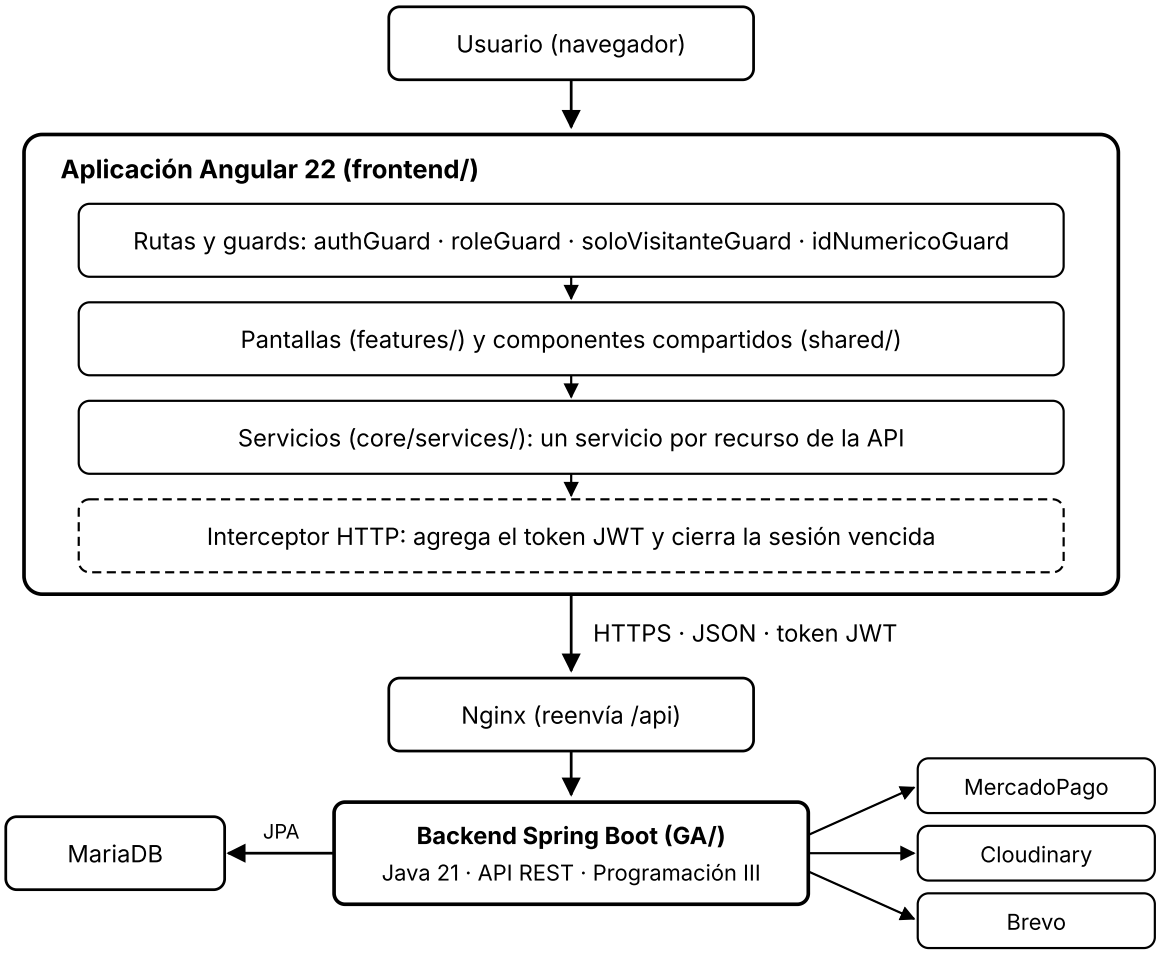


Figura 1. Arquitectura general: capas del frontend y su relación con el backend y los servicios externos.

3.2 Organización del código

El código del frontend se organiza en cuatro carpetas con responsabilidades separadas:

Carpeta	Contenido
`core/`	Lo que no se ve: configuración, guards, interceptor, modelos, servicios HTTP y utilidades (fechas, mensajes de error, reglas de cancelación).

Carpeta	Contenido
`shared/`	Piezas visuales reutilizables: carta de avisos, calendario de rango, gráfico de barras, mapa, visor de fotos, Kaza, esqueletos de carga y estados vacíos.
`layout/`	Encabezado, pie de página y páginas de error 403 y 404.
`features/`	Una carpeta por perfil (`public`, `auth`, `cliente`, `anfitrion`, `admin`) y, dentro, una carpeta por pantalla.

3.3 Principios de diseño del código

- **Componentes standalone y signals.** Cada pantalla guarda su estado en `signal()` y deriva el resto con `computed()`. Es la forma actual de Angular: evita suscripciones manuales y la vista se actualiza sola.
- **Un servicio por recurso.** Las pantallas nunca usan `HttpClient` directamente: piden los datos a `ReservaService`, `AlojamientoService`, etc. Un cambio en un endpoint se resuelve en un solo archivo.
- **Carga diferida.** Casi todas las pantallas se descargan recién cuando se visitan (`loadComponent`). Chart.js y Leaflet también se cargan a pedido. La carga inicial queda por debajo de 500 kB.
- **Configuración fuera del código.** La URL de la API se lee de `public/env.js`, de modo que el mismo build sirve en desarrollo (`http://localhost:8081`) y en el servidor (`/api`) sin recompilar.

4 Autenticación y protección de rutas

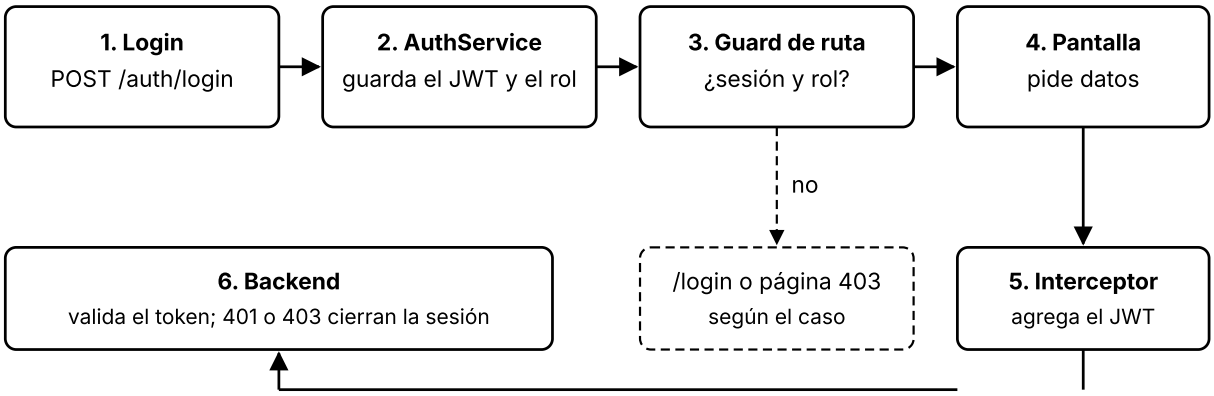


Figura 2. Flujo de autenticación y de una petición protegida.

1. **Inicio de sesión.** `POST /auth/login` devuelve un JWT. `AuthService` lo guarda, decodifica el rol y lo expone como signals (`estaLogueado()`, `rol()`, `usuario()`).
2. **Cada petición.** `authInterceptor` agrega el encabezado `Authorization: Bearer …`.
3. **Sesión vencida o cuenta desactivada.** Un temporizador cierra la sesión a la hora exacta en que vence el token. Si el backend responde 401, o 403 con un token que ya no es válido, se verifica la sesión con `/Usuario/me` y, si corresponde, se cierra con un aviso. De este modo, un usuario desactivado por el administrador queda fuera de inmediato.
4. **Guards.**
   - `authGuard`: exige sesión; si no la hay, lleva a `/login` y luego vuelve a la ruta pedida.
   - `roleGuard(['ADMIN'])`: exige un rol; si no coincide, muestra la página 403.
   - `soloVisitanteGuard`: quien ya inició sesión no accede al login ni al registro.
   - `idNumericoGuard`: una dirección como `/alojamientos/abc` lleva a "página no encontrada" en lugar de quedar cargando.
5. **Cambio de email.** El email es la identidad dentro del JWT. Si el usuario lo modifica, el backend devuelve un token nuevo en el encabezado `X-New-Token` y el frontend lo reemplaza sin cortar la sesión.

Además de los guards, las pantallas de edición verifican que el recurso pertenezca al usuario: un anfitrión no puede editar el alojamiento ni la galería de otro cambiando el número en la dirección (`core/utils/duenio.util.ts`). El backend, por su parte, rechaza cualquier operación sobre recursos ajenos.

5 Decisiones de diseño

5.1 Búsqueda y filtros en el navegador

El backend devuelve el catálogo completo y no acepta filtros por parámetros. En lugar de modificarlo, la búsqueda filtra en el navegador (texto sin distinguir tildes ni mayúsculas, precio, huéspedes y servicios) y pagina los resultados de a doce. Para filtrar por servicio se consultan una sola vez, en paralelo, los servicios de cada alojamiento. Los servicios se eligen en una carta aparte con buscador, porque el catálogo puede ser extenso.

5.2 Calendario de rango para reservar

Las fechas se eligen en un calendario propio (`shared/calendario-rango`): entrada y salida con los días intermedios marcados, y los días ocupados o pasados deshabilitados. Así se evita el error más frecuente, elegir fechas ya reservadas, antes de llegar al backend. El mismo componente se usa para elegir períodos en el flujo de dinero.

5.3 Fechas siempre en hora local

El método `toISOString()` convierte a UTC y, en Argentina (UTC-3), puede desplazar una fecha al día anterior. Todas las fechas pasan por `core/utils/fecha.util.ts`, que trabaja en hora local.

5.4 Una única carta para confirmaciones y avisos

No se usan los diálogos `alert()` ni `confirm()` del navegador. Toda confirmación ("¿Cancelar esta reserva?") y todo resultado ("Se te devuelven $X") se muestran en la misma carta (`shared/confirmacion`, mediante `ConfirmacionService`), con Kaza como sello. Antes de una acción que mueve dinero, la carta explica qué va a ocurrir; después, informa qué ocurrió. El foco inicial está en "Volver", para que una pulsación apresurada de Enter nunca ejecute la acción peligrosa.

5.5 Reglas de cancelación explicadas en pantalla

`core/utils/cancelacion.util.ts` replica las reglas del backend (reembolso total, retención del 35 % cuando el cliente cancela con menos de 48 horas, etc.) únicamente para **mostrar** el resultado antes de confirmar. El cálculo que vale es siempre el del backend.

5.6 Pagos simulados para la demostración

El entorno de prueba de MercadoPago no permite reembolsos ni liquidaciones a anfitriones. Con la variable `PAGOS_SIMULADOS=true`, el backend registra esos dos movimientos sin llamar a MercadoPago y las reservas cambian de estado igual. El frontend lo sabe por `env.js` (`pagosSimulados: true`) y lo aclara

en la carta con la leyenda "Transacción simulada". El cobro al cliente se realiza siempre a través de MercadoPago.

5.7 Baja de usuarios en cascada

El backend, al desactivar una cuenta, solo cambia su estado. Para que no queden alojamientos publicados de un anfitrión inactivo ni solicitudes de un cliente que ya no puede pagar, `BajaUsuarioService` muestra primero qué se verá afectado y, al confirmar, pausa los alojamientos y cancela las reservas sin pago. Las reservas ya pagadas no se modifican automáticamente, porque cancelarlas implica devolver dinero: quedan para que el administrador decida desde la ficha del usuario.

5.8 Ficha de usuario para el administrador

Desde la gestión de usuarios, el nombre de cada persona abre su ficha: números, alojamientos a su nombre (publicados o pausados) y reservas, con enlaces a cada alojamiento, cliente y anfitrión. En el listado general de reservas y en el detalle de cada reserva, esos datos también son enlaces. El administrador puede seguir cualquier recorrido sin búsquedas manuales.

5.9 Mensajes de error en lenguaje claro

El backend responde los errores en dos formatos y con lenguaje técnico. `http-error.util.ts` los normaliza y traduce los más comunes, por ejemplo: "Esas fechas ya están reservadas. Probá con otras." o "El usuario o la contraseña no son correctos."

5.10 Sin librerías de componentes

Se utiliza CSS propio, sin Angular Material, Bootstrap ni Tailwind. La identidad visual es propia y el peso de la aplicación es menor. Las únicas librerías agregadas son Chart.js (gráficos) y Leaflet (mapa).

6 Diseño visual

La identidad visual de AlojaME (paleta, logotipo, mascota, tipografía, componentes y tono de voz) se documenta en detalle en el *Manual de marca*. En síntesis:

- **Paleta:** tinta casi negra, tomate, dorado, azul y acero, definidos una sola vez como variables en `src/styles.css`. Ninguna pantalla usa colores sueltos.
- **Tipografía:** Plus Jakarta Sans para textos y títulos; Jaybird solo para el logotipo.
- **Kaza:** la mascota, una casita con cara. Aparece en el logotipo, en el avatar del usuario, en las páginas 403 y 404 y como insignia de la carta de avisos, donde entra con una breve animación y se balancea sobre el centro de su cara.
- **Gráficos (Chart.js):** pensados para quien no está habituado a leer gráficos. Cada uno muestra arriba una frase con la conclusión ("X encabeza con $Y, el 40 % del total"), el valor escrito en cada barra y la barra más alta resaltada. Un botón "i" junto al título abre la explicación de qué muestra el gráfico y cómo se calcula, y "Ver los datos" presenta los mismos números en una tabla.
- **Estados de carga y vacíos:** esqueletos mientras se cargan los datos y mensajes con una acción cuando no hay resultados ("Limpiar filtros", "Publicar alojamiento").

7 Diseño responsive

El diseño es *mobile-first*, con tres cortes: base para celular (probada en 360 píxeles), tablet desde 768 píxeles y escritorio desde 1024 píxeles. Las utilidades globales de `src/styles.css` resuelven los casos repetidos:

Utilidad	Comportamiento
`.form-grid`	Formularios en una columna en el celular y en dos desde tablet.
`.table-cards` con `data-label`	Las tablas se transforman en tarjetas por debajo de 1024 píxeles.
`.kpi-grid`	Tarjetas de indicadores que se reacomodan según el ancho disponible.

Todas las pantallas se verificaron con los cuatro perfiles en 360, 768, 1024 y 1280 píxeles: ninguna presenta desplazamiento horizontal y todos los controles miden al menos 44 píxeles de alto.

8 Limitaciones conocidas

- **Liquidaciones de MercadoPago:** la API de liquidaciones rechaza las credenciales de prueba. Para la demostración se usa el modo de pagos simulados (sección 4.6).
- **Funciones que requieren cambios en el backend:** búsqueda por disponibilidad de fechas, recuperación de contraseña, cantidad de huéspedes por reserva y avisos por correo de las reservas. Ninguna impide el funcionamiento actual: el backend no expone esos datos ni esas operaciones, y quedan como mejoras a futuro.
- **Pruebas automatizadas:** el frontend no tiene pruebas unitarias. Se verificó con recorridos completos por cada perfil contra el backend.

9 Cómo probarlo

1. Levantar el backend y el frontend como indica el *README* del repositorio.
2. Cargar los datos de demostración: `GA/db/datos_demo_completa.sql`. Al terminar, el script imprime una guía con cada cuenta de prueba y el caso que permite mostrar (reserva por pagar, cancelación tardía, reserva para finalizar, usuario desactivado, entre otros).
3. Para la demostración, activar `PAGOS_SIMULADOS=true` en el backend y `pagosSimulados: true` en `env.js`.